PulsePoll NextGen Real-Time Live Polling Platform

Full-Stack Project Documentation, System Architecture & Submission Dossier

Tech Stack: React 18 | Go 1.22 (Gin) | Redis 7 (Pub/Sub + Counters) | MongoDB 6

Prepared for: HCL GUVI Developer Task | Submission: dev@hclguvi.com

1. Project Overview & Problem Statement

PulsePoll is a production-grade, real-time live polling platform built for high-concurrency audience interaction. A presenter creates a poll, distributes the link or QR code, and audience members cast their votes from their phones or browsers. Every connected presenter and audience member sees the percentage bars, vote counts, and winner highlights update instantaneously with zero page refreshes.

The project strictly complies with the HCL GUVI Developer Task specifications, implementing complete separation of concerns between the React frontend and Go backend, server-side input validation, atomic Redis operations, and durable MongoDB storage.

2. Required Tech Stack & Deep Architectural Decisions

All four required layers are doing high-value architectural work rather than serving as boilerplate:

- **Frontend (React 18 + Vite + Tailwind): Next-generation glassmorphic dark theme,** real-time animated percentage bars, live viewer counter, celebratory confetti animations, QR code generator, and Presenter mode.
- **Backend (Go 1.22 + Gin): High-performance compiled API gateway, Gorilla WebSockets with** auto-reconnection/heartbeat, Bcrypt password hashing, JWT token authorization, and strict input validation.
- **Realtime Engine (Redis 7): Atomic HINCRBY prevents race conditions during concurrent** voting bursts; SADD set checks enforce duplicate vote prevention at O(1); Pub/Sub broadcasts vote events across server instances; INCR/DECR tracks active audience presence.
- **Persistence (MongoDB 6): Stores user accounts, poll definitions, option metadata, and an** immutable votes audit log with voter fingerprints, timestamps, IPs, and user-agents for CSV export.

Real Work Done by Redis vs MongoDB (Architectural Justification)

A common anti-pattern in live polling tools is writing every incoming vote directly to a disk-based document database. Under bursts of hundreds of concurrent votes, database write locks cause latency spikes and race conditions.

In PulsePoll, Redis acts as the real-time shock absorber: Redis HINCRBY executes tallies in-memory within microseconds, SADD verifies voter uniqueness instantly, and Redis Pub/Sub

fans out the updated tally across all connected WebSockets. MongoDB persistence is handled asynchronously in a background goroutine for audit logging, ensuring the voter never experiences latency.

3. How to Run the Project Locally

You can run PulsePoll locally either with Docker Compose (single command) or directly on macOS:

Option A: 1-Click Docker Compose (Recommended)

```
# Navigate to project root and launch all 4 services:
cd /Users/akhil/.gemini/antigravity/scratch/pulsepoll
docker compose up --build

# Access points:
Frontend UI: http://localhost:3000  |  Go Backend API: http://localhost:8080
```

Option B: Native macOS Execution

```
# 1. Start Redis (Port 6379):
brew services start redis

# 2. Start Go Backend (Port 8080) in Terminal 1:
cd /Users/akhil/.gemini/antigravity/scratch/pulsepoll/backend
go run main.go

# 3. Start React Frontend (Port 5173) in Terminal 2:
cd /Users/akhil/.gemini/antigravity/scratch/pulsepoll/frontend
npm run dev
```

4. Generating the Live Public Host Link

The specification states: 'Actually deployed. Ship it to a live, working link, not just localhost.' You have two simple options:

- **Instant Public Live Link (Tunneling): Run 'npx localtunnel --port 3000' or 'cloudflared** tunnel --url http://localhost:3000' while running locally to obtain a publicly reachable HTTPS URL accessible from any phone or browser worldwide.
- **Permanent Cloud Deployment (Render): Push this repository to GitHub, link it on** Render.com using the included render.yaml blueprint, and plug in free-tier MongoDB Atlas and Upstash Redis URIs.

5. Mandatory 3-5 Minute Submission Video Script

[IMPORTANT] Recording the 3-5 minute video is mandatory per task instructions. Email to dev@ hclguvi.com.

Section 1: Live Demo (1.5 Minutes)

Walk through the user journey: Log in using the 1-Click Demo Login button

(demo@pulsepoll.io). Create a new poll with 3-4 options. Open the poll link in two side-by-side browser windows. Cast a vote in Window A and show Window B updating instantaneously with smooth animated bars and confetti. Show the QR code modal and CSV export.

Section 2: The Challenge That Gave the Most Trouble (1.5 Minutes)

Script recommendation: 'The biggest architectural challenge was handling high-frequency concurrent voting without write-lock contention or race conditions. If multiple voters submit votes in the same millisecond, traditional database read-modify-write patterns suffer from collisions. We solved this by decoupling the real-time counting from database persistence: Redis atomic HINCRBY tallies votes in sub-milliseconds, SADD checks duplicate voters at O(1), and Redis Pub/Sub pushes updates across WebSockets. MongoDB persistence is handled asynchronously in background goroutines.'

Section 3: How AI Tools Were Used (1 Minute)

Script recommendation: 'Yes, I utilized AI coding assistance (Antigravity with Gemini) during development. The AI was particularly valuable for rapid scaffolding of Go Gin handlers, crafting the Tailwind glassmorphic styling system, and generating Docker orchestration files. However, the core distributed architecture--designing the atomic Redis data structures, implementing the WebSocket hub lifecycle with viewer presence counting, and building the zero-crash DataStore fallback--required careful human validation and architectural rigor.'

6. Key Source Code Highlights

Atomic Redis Engine (backend/database/redis.go)

```
func (r *RedisService) IncrementVote(ctx context.Context, pollID string, optionIndex int) (int64, error) {
    return r.Client.HIncrBy(ctx, fmt.Sprintf("poll:%s:votes", pollID), strconv.Itoa(optionIndex), 1).Result()
}
func (r *RedisService) RecordVoter(ctx context.Context, pollID, voterID string) error {
    return r.Client.SAdd(ctx, fmt.Sprintf("poll:%s:voters", pollID), voterID).Err()
}
```

WebSocket Hub & Redis Pub/Sub Relay (backend/websocket/hub.go)

```
pubsub := redis.Client.Subscribe(ctx, redis.PubSubChannel(pollIDHex))
defer pubsub.Close()
for redisMsg := range pubsub.Channel() {
    var payload models.WebSocketMessage
    json.Unmarshal([]byte(redisMsg.Payload), &payload)
    conn.WriteJSON(payload) // Stream live updates to browser without refresh
}
```

7. Submission Checklist for HCL GUVI

- **1. Public GitHub Repository Link: Full source code, git history, separation of concerns** (/frontend, /backend), README.md.
- **2. Working Live Deployed Link: Publicly reachable link generated via Render, Railway, or**

Cloudflare/localtunnel.

- **3. Short Video (3-5 min): Unlisted YouTube or Google Drive link answering the** concurrency challenge and AI tool usage.
- **4. Email Destination: Send all 3 links to: dev@hclguvi.com.**